SPRING SECURITY / HANDS-ON EXERCISE

Authorization in a Training Room Booking API

Endpoint-level and method-level security | Student handout

Scenario

A company provides training rooms that employees can reserve for workshops. You are asked to secure an existing Spring Boot REST API. The application already has controllers, services, sample rooms and reservations, and a working authentication mechanism. Your task is to implement authorization only.

Learning objectives

- Configure HTTP request authorization with SecurityFilterChain and request matchers.
- Differentiate Spring Security roles from authorities.
- Enable method security and write @PreAuthorize expressions.
- Use the current Authentication and method arguments in access rules.
- Delegate resource-specific checks to a Spring-managed security bean.
- Explain the responsibility of endpoint checks versus service checks.

Assumptions and boundaries

Authentication is already configured. Users have the roles and authorities specified below. You do not need to implement sign-in, users/passwords, tokens, persistence or a UI. Implementing the business logic is not the objective.

API overview

GET	/api/rooms	List available rooms
POST	/api/reservations	Create a room reservation
GET	/api/reservations/{id}	View reservation details
DELETE	/api/reservations/{id}	Cancel a reservation
GET	/api/admin/usage	View room utilization report

Roles: EMPLOYEE, COORDINATOR, ADMIN. Additional authority: ROOM_APPROVE. An authenticated account has at least one of the listed roles.

Tasks

Requirement 1 - Endpoint-level authorization

Configure a SecurityFilterChain using authorizeHttpRequests() and request matchers so that:

• GET /api/rooms is public.

• /api/admin/** requires the ADMIN role.

• POST /api/reservations requires authentication.

• All other /api/** endpoints require authentication.

• Every request not covered above is denied.

Requirement 2 - Enable method security

Enable Spring Security method-level authorization. Add the appropriate annotation to the application configuration.

Requirement 3 - Coordinator-only operation

The reservation service contains:

```
public void cancelAnyReservation(Long reservationId) {
    // business logic already provided
}
```

Only COORDINATOR and ADMIN users may call this method. Apply method-level authorization to the service method.

Requirement 4 - Authority-based approval

A service operation approves the use of a room:

```
public void approveReservation(Long reservationId) {
    // business logic already provided
}
```

Only users having the ROOM_APPROVE authority may invoke it. This requirement must not depend on a role.

Requirement 5 - Reservation ownership

The service exposes:

```
public Reservation getReservation(Long reservationId) {
    // business logic already provided
}
```

A reservation may be read only if the caller either owns that reservation or has the ADMIN role. The project contains the following component:

```
@Component
public class ReservationSecurity {
    public boolean isOwner(Long reservationId,
                           Authentication authentication) {
        // returns whether this user owns the reservation
    }
}
```

Write a @PreAuthorize expression on getReservation() that calls this bean, passing the reservation ID and the current authentication.

Requirement 6 - HTTP checks and service checks

The controller delegates requests as follows:

```
@GetMapping("/api/reservations/{id}")
public Reservation getReservation(@PathVariable Long id) {
    return reservationService.getReservation(id);
}
```

Require authentication at the HTTP level for GET /api/reservations/{id}, while keeping the ownership-or-admin check at the service level.

Short answer (2-4 sentences): Why are both checks useful? Explain what each protects, including what happens if another component calls the service directly.

Bonus challenge - Combine roles and method arguments

Add a service operation:

```
public void changeReservationStatus(Long reservationId,
                                    String status) {
    // business logic already provided
}
```

Implement the following entire policy in one @PreAuthorize SpEL expression:

- A COORDINATOR may set the status to CONFIRMED or COMPLETED.
- Only an ADMIN may set the status to REJECTED.
- No other role/status combinations are allowed, including unknown statuses.

Assume that status values are uppercase strings. You may reference the method parameter in SpEL.

What to submit

- Security configuration with HTTP authorization rules.
- Configuration enabling method security.
- Service method annotations for requirements 3-5 and, optionally, the bonus.
- A short explanation for requirement 6.

Suggested checks

- Anonymous request to GET /api/rooms is allowed; anonymous reservation creation is rejected.
- Authenticated EMPLOYEE cannot access /api/admin/usage.
- A user lacking ROOM_APPROVE cannot call approveReservation(), regardless of role.
- A non-owner who is not ADMIN cannot read another employee's reservation.
- The bonus denies COORDINATOR + REJECTED and ADMIN + unknown status.

Reminder: Endpoint-level authorization uses HTTP request information. Method security protects service operations, including calls that do not originate from the mapped HTTP controller.